

RAPIDSEGMENT

Transparent segments. Production-ready SQL.
No black-box trees.

Strategic Segmentation & Scorecard Engine



RapidSegment

FAST. INTELLIGENT. HIGH-IMPACT SEGMENTS.

A fast, multi-segment rule discovery and
customer segmentation engine for tabular data.



FAST
& SCALABLE



HIGH IV
SEGMENTS



AUTOMATED
RULE DISCOVERY



INTERPRETABLE
& TRANSPARENT

The problem with decision trees



Hard to load

Wide tables, mixed sources,
memory spikes



Encoding tax

One-hot / label encode kills
meaning



Unreadable

Past depth 4 no one can
explain it



Same root bias

Always starts from the same
variable

DECISION TREE vs RAPIDSEGMENT

A fundamental shift from traditional splitting to stakeholder-centric, robust segmentation.

Key Capability	Decision Tree (DT)	RapidSegment
 Splitting Dynamics	— Splits on 1 variable at a time (univariate).	✓ Can choose to split up to 3 variable interactions simultaneously.
 Feature Reuse	— Splits on the same variable at various depths, reducing explainability to stakeholders.	✓ Provides explicit options to reduce feature reuse, keeping rules clean.
 Segment Uniqueness	— Complementary (Yes/No) branches using similar variables; creates overlapping contexts.	✓ Groups similar variables to select only the best, yielding highly distinct, unique segments.
 Business Constraints	— Maximizes Gini Gain; only provides basic controls for minimum sample size.	✓ Guarantees hard-defined min event, min lift, and min capture per segment.
 NULL Handling	— Requires pre-imputation of missing values or relies on complex surrogate splits.	✓ Natively handles NULLs directly within the SQL output without preprocessing.
 Output Format	— Yields complex, deeply nested IF-ELSE tree paths that are difficult to audit.	✓ Produces flat, audit-ready ANSI SQL rules ready for immediate deployment.

Hours vs minutes



Decision Tree path

1. Load & clean
2. Encode categoricals
3. Fit tree
4. Extract leaves manually
5. Residual loops ×4
6. Translate to SQL

4–12+ hours



RapidSegment path

1. Load with UniversalDataLoader
2. Native categoricals (no encode)
3. `extract_segments()` — one call
4. SQL filters produced
5. Scorecard JSON + weights

5–15* minutes

**Depends on the size of data but mostly faster as tested on 2.5M dataset and 30 columns*

What stakeholders actually see

Decision Tree — not human friendly

```
max_dpd_12m ≤ 15.5
├─ util_avg ≤ 0.42
│   ├─ risk_seg = Low → Leaf A
│   └─ util_avg > 0.42 → Leaf C/D
└─ max_dpd_12m > 15.5
    ├─ util ≤ 0.71 → Leaf E/F
    └─ ... 47 more nodes ...
        └─ [unreadable]
```

RapidSegment — readable SQL

SEG 1 Lift 4.8× n=2,140

```
max_dpd_12m >= 60
AND utilization >= 0.85
```

SEG 2 Lift 3.1× n=5,820

```
risk_segment = 'High'
AND max_dpd_12m >= 30
```

SEG 3 Lift 2.4× n=3,450

```
utilization >= 0.90
AND tenure_months < 12
```

Four building blocks



BigQueryFeatureSelector

Screen IV in-cloud
Download only winners



UniversalDataLoader

CSV · Parquet · Excel
Arrow · Big Query



StrategicSegmentBuilder

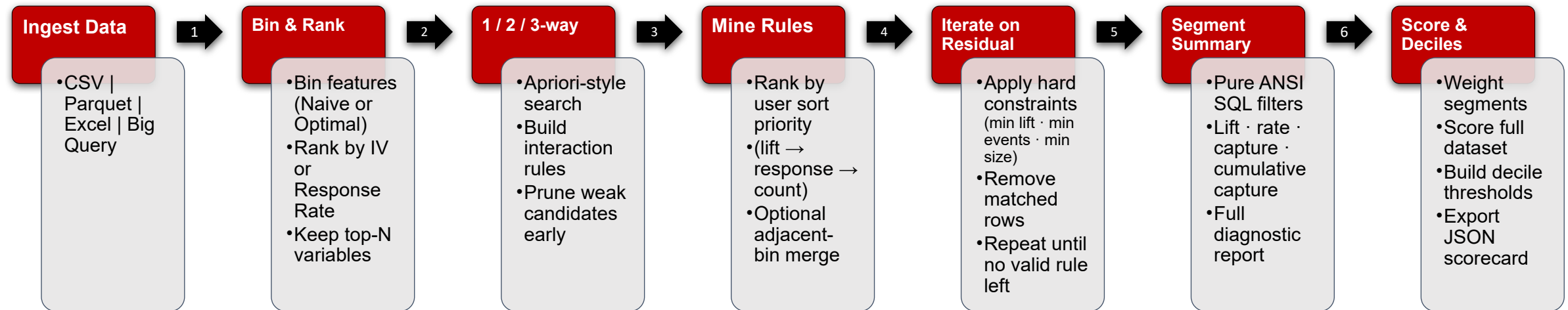
Finds high-lift rules
Outputs pure SQL



StrategicSegmentScore

Weights + deciles
JSON scorecard

How it works — 6 steps



How we rank features — IV & WOE (simple)

WOE — Weight of Evidence

In plain English

How much more (or less) likely an event is in this group compared with the average.

Formula

$$\text{WOE} = \ln\left(\frac{\% \text{ of non-events}}{\% \text{ of events}} \right)$$

Positive WOE → safer group Negative WOE → riskier group

IV — Information Value

In plain English

How useful is this whole feature at separating events from non-events?

Formula

$$\text{IV} = \sum \left(\% \text{non-events} - \% \text{events} \right) \times \text{WOE}$$

Higher IV = stronger predictor

Quick example — max_dpd_12m

Bin	% Events	% Non-events	WOE	Contribution
0 days	20%	55%	+1.01	Strong (safer)
30–59 days	25%	25%	0.00	Neutral
60+ days	55%	20%	–1.01	Strong (riskier)

IV ≈ 0.70 → Strong predictor (rule of thumb: IV > 0.30 is useful)

Why teams choose it



No encoding

Categoricals stay categorical



Multi-way rules

1-, 2-, 3-way via Apriori



Zero overlap

Hierarchical residual guarantees it



ANSI SQL

Teams & production share one language



Cloud-first

Screen IV inside Big Query



Full audit

- Explain why a feature was selected/not selected at each step

Just 5 lines of code to build Interpretable Segments

```
from rapidsegment import StrategicSegmentBuilder, StrategicSegmentScore

#Configure Experiment
builder = StrategicSegmentBuilder(target="default_flag",max_segments=10)

#Run Experiment – Build Segment
segments = builder.extract_segments(data)

#View Segments
final_eval = builder.evaluate_final_coverage(data)

#Score on dataset
scorer = StrategicSegmentScore(
    target_col="default_flag", primary_key="cust_id",
    segment_cols=[...])

#Export Scores
model = scorer.calculate_and_export_weights(df, "model.json")
```

Diagnostics & Explainability

Complete audit trail for every extraction run

explain_feature_journey(feature)

Full audit trail of a feature across every residual iteration — why it was kept, dropped, or reused.

explain_no_segments()

Human-readable report explaining why extraction stopped early or returned zero segments.

generate_feature_health_report()

Binned feature that determines response concentration. Useful in cases where feature variance is low or has NULL entries

diagnostics_ + candidate funnel

Log showing Per-iteration record of candidates, near-misses, winning segment, and expansion stats.

Stop explaining black boxes. Start shipping readable rules.

High-lift segments → pure SQL → calibrated scorecard.
Risk, credit, and marketing can all act with confidence.

github.com/B2BDA/RapidSegment · `pip install rapidsegment`



RapidSegment

FAST. INTELLIGENT. HIGH-IMPACT SEGMENTS.

A fast, multi-segment rule discovery and
customer segmentation engine for tabular data.



FAST
& SCALABLE



HIGH IV
SEGMENTS



AUTOMATED
RULE DISCOVERY



INTERPRETABLE
& TRANSPARENT